

Week 5 – PySpark Assignment Report

Name: Gaurav Kumar

Internship: Celebal Technologies

Overview

This report contains concise answers for the Week 5 PySpark assignment. The answers summarize the concepts and include representative PySpark code where required. Execution screenshots can be added before converting the document to PDF.

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

MapReduce relies heavily on disk I/O, making iterative processing slow. It follows a rigid Map and Reduce model and requires multiple jobs for complex workflows. Apache Spark performs in-memory computation, uses DAG optimization, supports SQL, streaming, machine learning, and graph processing, making it faster and more flexible.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Spark stores intermediate data in RAM instead of repeatedly reading and writing to disk. Machine learning algorithms process the same dataset multiple times, so caching data in memory significantly reduces execution time compared to disk-based systems like MapReduce.

Q3. Remove duplicate rows based on user_id and transaction_date.

```
df_clean = df.dropDuplicates(["user_id", "transaction_date"])
```

Screenshot:

user_id	transaction_date	region	product_category	sale_amount	price	city	status	email	username	age	subscription
1001	2025-01-20	West	Clothing	8101.5	1350.25	Pune	NULL	user1001@example.com	user_1001	21	Bas
1001	2025-02-22	North	Electronics	11399.95	2279.99	Pune	Pending	user1001@example.com	user_1001	25	Standar
1001	2025-03-21	South	Grocery	3838.71	3838.71	Hyderabad	Completed	user1001@example.com	user_1001	35	Bas
1002	2025-01-11	South	Electronics	288.14	144.07	Mumbai	Cancelled	user1002@example.com	user_1002	23	Bas
1002	2025-02-01	North	Sports	16729.32	4182.33	Bengaluru	Pending	user1002@example.com	user_1002	18	Bas
1002	2025-02-09	East	Electronics	6754.32	2251.44	Delhi	Pending	user1002@example.com	user_1002	29	Bas
1002	2025-02-10	North	Clothing	2537.14	1268.57	Hyderabad	Pending	user1002@example.com	user_1002	37	Standar
1002	2025-02-20	South	Grocery	1255.12	627.56	Pune	NULL	user1002@example.com	user_1002	25	Standar
1002	2025-03-15	South	Furniture	13182.9	2636.58	Delhi	Cancelled	user1002@example.com	user_1002	29	Bas
1002	2025-03-17	South	Grocery	0.0	NULL	Bengaluru	Cancelled	user1002@example.com	user_1002	30	Premi
1003	2025-01-02	West	Clothing	13547.24	3386.81	Chennai	Cancelled	user1003@example.com	user_1003	35	Bas
1003	2025-01-13	East	Furniture	19613.7	3922.74	Mumbai	Cancelled	user1003@example.com	user_1003	16	Premi
1003	2025-01-14	West	Furniture	0.0	NULL	Bengaluru	Pending	user1003@example.com	user_1003	31	Standar
1003	2025-02-12	North	Electronics	343.44	57.24	Bengaluru	Pending	user1003@example.com	user_1003	32	Premi
1003	2025-02-24	South	Grocery	20407.86	3401.31	Delhi	Completed	user1003@example.com	user_1003	28	Premi
1003	2025-03-01	East	Electronics	27065.94	4510.99	Chennai	NULL	user1003@example.com	user_1003	27	Standar
1003	2025-03-07	North	Clothing	13826.58	2304.43	Pune	Pending	user1003@example.com	user_1003	25	Standar
1003	2025-03-18	West	Grocery	7172.44	1793.11	Chennai	Pending	NULL	user_1003	31	Premi
1003	2025-03-22	West	Sports	7723.32	3861.66	Delhi	NULL	user1003@example.com	user_1003	16	Standar
1003	2025-03-29	North	Clothing	21705.42	3617.57	Pune	Completed	user1003@example.com	user_1003	25	Bas

Q4. Filter region='West' and find average sale_amount by product_category.

from pyspark.sql.functions import avg

result = (df.filter(df.region=="West")

.groupBy("product_category")

.agg(avg("sale_amount").alias("average_sale_amount")))

Screenshot:

product_category	average_sale_amount
Sports	8162.618636363637
Grocery	10224.180399999999
Electronics	8697.399090909092
Clothing	8235.07575757576
Furniture	9190.613953488371

Q5. Difference between .na.drop() and .na.fill().

.na.drop() removes rows containing null values, whereas .na.fill() replaces null values with a specified value.

Example:

df_clean = df.na.fill("Unknown", subset=["status"])

Screenshot:

```

+-----+
|  status |
+-----+
| Pending |
| Pending |
| Cancelled |
| Completed |
| Completed |
| Cancelled |
| Pending |
| Unknown |
| Unknown |
| Unknown |
| Completed |
| Pending |
| Cancelled |
| Pending |
| Unknown |
| Completed |
| Unknown |
| Completed |
| Unknown |
| Unknown |
+-----+
only showing top 20 rows

```

Q6. Count records for each city where count > 100.

```

from pyspark.sql.functions import count
result = (df.groupBy("city")
           .agg(count("*").alias("total_records"))
           .filter("total_records > 100"))

```

Screenshot:

```

+-----+-----+
|  city |total_records |
+-----+-----+
| Delhi |          262 |
| Mumbai |          216 |
| Bengaluru |          185 |
| Hyderabad |          148 |
| Pune |          118 |
+-----+-----+

```

Q7. How does DataFrame immutability affect data cleaning?

Spark DataFrames are immutable. Operations such as dropping columns, renaming columns, or filtering data never modify the original DataFrame. Instead, each transformation returns a new DataFrame, improving fault tolerance and enabling Spark to optimize execution.

Q8. Filter age between 18 and 30 and subscription='Premium'.

```
from pyspark.sql.functions import col
result = df.filter((col("age").between(18,30)) &
                  (col("subscription")=="Premium"))
```

Screenshot:

user_id	transaction_date	region	product_category	sale_amount	price	city	status	email	username	age	subscription
1115	2025-03-17	South	Clothing	1427.04	1427.04	Mumbai	Cancelled	user1115@example.com	user_1115	20	Premium
1064	2025-03-13	South	Sports	16307.64	2717.94	Chennai	NULL	user1064@example.com	user_1064	27	Premium
1136	2025-01-01	West	Clothing	12059.12	3014.78	Delhi	Cancelled	user1136@example.com	user_1136	23	Premium
1139	2025-03-30	North	Grocery	3137.1	1045.7	Hyderabad	NULL	user1139@example.com	user_1139	19	Premium
1050	2025-01-25	East	Clothing	5409.22	2704.61	Hyderabad	NULL	user1050@example.com	user_1050	23	Premium
1098	2025-01-01	North	Grocery	14781.84	4927.28	Hyderabad	NULL	user1098@example.com	user_1098	20	Premium
1086	2025-02-22	North	Furniture	24126.36	4021.06	Mumbai	Completed	user1086@example.com	NULL	28	Premium
1112	2025-02-14	West	Grocery	10968.9	3656.3	Hyderabad	Pending	user1112@example.com	user_1112	24	Premium
1122	2025-02-27	South	Furniture	2666.0	1333.0	Bengaluru	Cancelled	user1122@example.com	user_1122	30	Premium
1139	2025-01-07	West	Electronics	14998.62	4999.54	Mumbai	Completed	user1139@example.com	user_1139	23	Premium
1004	2025-02-23	North	Grocery	4120.54	4120.54	Hyderabad	Pending	user1004@example.com	user_1004	29	Premium
1026	2025-01-25	North	Electronics	4549.26	1516.42	Mumbai	Pending	user1026@example.com	user_1026	24	Premium
1065	2025-03-03	South	Electronics	614.73	614.73	Hyderabad	Completed	user1065@example.com	NULL	20	Premium
1016	2025-03-20	East	Clothing	4802.54	4802.54	Mumbai	Completed	user1016@example.com	user_1016	21	Premium
1008	2025-01-08	West	Grocery	19972.0	4993.0	Hyderabad	NULL	user1008@example.com	user_1008	18	Premium
1025	2025-02-10	North	Furniture	12974.88	2162.48	Mumbai	NULL	user1025@example.com	user_1025	18	Premium
1074	2025-03-18	East	Furniture	1596.62	1596.62	Chennai	NULL	user1074@example.com	user_1074	23	Premium
1020	2025-02-10	South	Clothing	14500.05	2900.01	Hyderabad	Completed	user1020@example.com	user_1020	28	Premium
1068	2025-03-16	West	Sports	11859.51	3953.17	Mumbai	Cancelled	NULL	user_1068	20	Premium
1052	2025-01-06	South	Sports	4836.6	1612.2	Pune	NULL	user1052@example.com	user_1052	24	Premium

only showing top 20 rows

Q9. Why handle null values before aggregations?

Null values can produce inaccurate totals and averages or lead to misleading reports. Cleaning null values before aggregation improves data quality and ensures reliable analytical results.

Q10. Cast raw_timestamp to TimestampType and rename it to event_time.

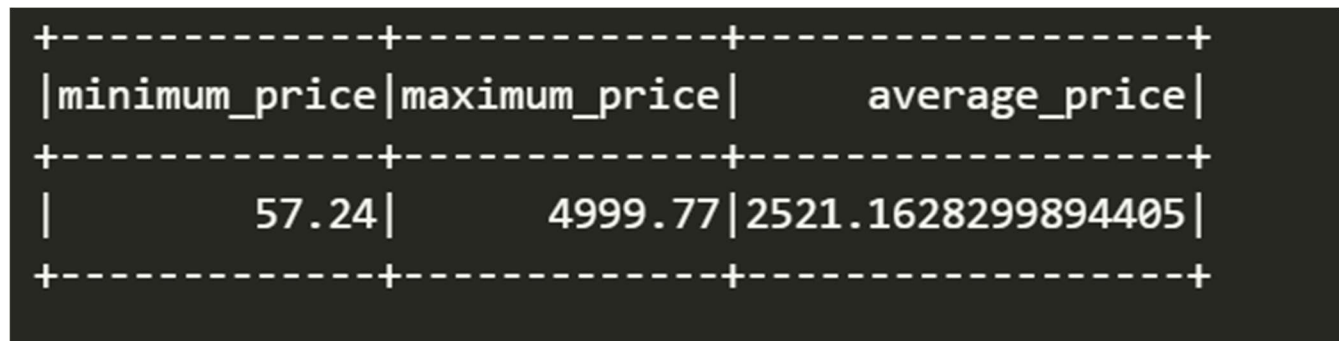
```
from pyspark.sql.functions import col
from pyspark.sql.types import TimestampType
```

```
df_updated = (df.withColumn("raw_timestamp",
                             col("raw_timestamp").cast(TimestampType()))
              .withColumnRenamed("raw_timestamp", "event_time"))
```


Q13. Calculate min, max, and average price using agg().

```
from pyspark.sql.functions import min,max,avg
result = df.agg(
    min("price").alias("minimum_price"),
    max("price").alias("maximum_price"),
    avg("price").alias("average_price")
)
```

Screenshot:



minimum_price	maximum_price	average_price
57.24	4999.77	2521.1628299894405

Q14. Risk of using inferSchema=true with inconsistent date formats.

When source data contains inconsistent date formats, Spark may infer the wrong data type (often StringType instead of DateType). This can cause parsing errors, incorrect filtering, and unreliable analytics. In production, defining an explicit schema is considered a best practice.

Q15. Build a processing pipeline.

```
from pyspark.sql.functions import sum

result = (df
    .dropDuplicates(["user_id","transaction_date"])
    .na.fill({"price":0})
    .groupBy("store_id")
    .agg(sum("price").alias("total_revenue"))
    .orderBy("total_revenue", ascending=False)
)
```

Screenshot:

```

+-----+-----+
|store_id|total_revenue|
+-----+-----+
|      12|174116.36000000002|
|      15|168745.29000000004|
|      14|166790.80000000005|
|       6|162820.89999999994|
|       2|157627.44999999995|
|       3|      156851.83|
|       1|154580.61000000002|
|      13|151790.68000000002|
|       9|150827.91000000003|
|       4|145072.16999999995|
|      11|      143954.61|
|       7|140004.32999999996|
|      10|135971.16999999998|
|       5|132851.74000000002|
|       8|      131975.02|
+-----+-----+

```

Key Learnings

- Understood Apache Spark fundamentals.
- Learned DataFrame transformations and actions.
- Performed data cleaning using duplicate removal and null handling.
- Applied filtering, grouping, aggregation, and type conversion.
- Understood shuffle, immutability, and schema inference.
- Built a simple end-to-end PySpark processing pipeline.

Conclusion

Through this assignment, practical experience was gained in using PySpark for scalable data processing. The concepts covered—including filtering, aggregation, data cleaning, and pipeline development—form a strong foundation for modern Data Engineering and ETL workflows.